

CAHIER DES CHARGES

Systeme de Banque en Ligne

Date:	10/06/2026
Version:	2.1 - Corrigée
Type:	Projet Academique
Backend:	Symfony 7 (CLI 5.17.1)
Frontend:	Twig + Bootstrap
Base de donnees:	PostgreSQL
Tables:	Banque, Client, Compte, Transaction

TABLE DES MATIÈRES

1. Présentation du projet
2. Objectifs spécifiques
3. Périmètre fonctionnel
4. Besoins fonctionnels
5. Besoins non-fonctionnels
6. Règles métier
7. Architecture technique
8. Modèle de données (4 tables)
9. Diagrammes UML
10. Livrables

1. Présentation du projet

Contexte

Dans le cadre d'un projet académique, il est demandé de concevoir et développer une plateforme web permettant à des banques de gérer leurs clients et leurs comptes bancaires à travers une interface web sécurisée.

Objectif général

Développer une plateforme multi-banque permettant aux banques de gérer leurs clients, aux clients de consulter leurs comptes et d'effectuer des virements, et à un administrateur de superviser l'ensemble du système.

2. Objectifs spécifiques

- Permettre aux banques de créer un compte et de gérer leurs clients
- Permettre aux clients de créer un compte et de choisir leur banque
- Permettre aux clients de consulter leurs comptes et soldes
- Permettre aux clients d'effectuer des virements internes sécurisés
- Permettre à l'administrateur de gérer les banques et les statistiques globales
- Permettre aux banques de gérer l'activation/désactivation des comptes clients

3. Périmètre fonctionnel

Espace Admin

- Gestion des banques (création, activation/désactivation)
- Consultation des statistiques globales

Espace Banque

- Gestion des clients (activation/désactivation de comptes)
- Consultation des clients et leurs comptes
- Consultation des transactions de ses clients
- Consultation de ses statistiques

Espace Client

- Inscription et sélection de sa banque
- Authentification sécurisée (connexion/déconnexion)
- Modification du mot de passe
- Consultation des comptes et soldes
- Effectuer un virement interne
- Consultation de l'historique des transactions
- Recherche et filtrage des opérations

4. Besoins fonctionnels

Gestion des banques

- Créer une banque
- Activer/désactiver une banque
- Consulter la liste des banques
- Modifier les informations d'une banque

Gestion des clients

- Créer un client (inscription)
- Modifier un client
- Supprimer ou désactiver un client
- Gérer l'association client-banque

Gestion des comptes

- Créer un compte bancaire
- Consulter le solde et les informations du compte
- Activer/désactiver un compte
- Mettre à jour les informations du compte

Gestion des transactions

- Effectuer un virement entre comptes
- Vérifier le solde disponible
- Enregistrer automatiquement les opérations
- Générer l'historique des transactions
- Gérer les statuts de transaction (succès/échec)

5. Besoins non-fonctionnels

Sécurité

- Chiffrement des mots de passe (bcrypt/Argon2)
- Contrôle d'accès basé sur les rôles (RBAC) : ROLE_ADMIN, ROLE_BANQUE, ROLE_CLIENT
- Validation des données saisies côté serveur
- Protection contre les attaques CSRF, XSS, SQL Injection
- HTTPS obligatoire en production

Performance

- Temps de réponse inférieur à 3 secondes
- Pagination des listes longues
- Indexation des bases de données

Disponibilité et Ergonomie

- Application accessible 24h/24 en environnement de test
- Interface simple et intuitive
- Compatible ordinateur et mobile (responsive design)
- Messages d'erreur clairs et explicites

6. Règles métier

Règle	Description	Impact
R1	Un virement est refusé si le solde du compte source est insuffisant. La transaction est refusée avec le statut ECHOUÉ.	Protection des données
R2	Un compte désactivé ne peut ni émettre ni recevoir de virement.	Contrôle d'accès au compte
R3	Le solde d'un compte ne peut jamais être négatif.	Intégrité des données
R4	Un virement réussi crée 2 transactions: débit sur le compte source, crédit sur le compte de destination.	Cohérence comptable

7. Architecture technique

Composant	Technologie	Version
Backend	Symfony Framework	7
CLI	Symfony CLI	5.17.1
Frontend	Twig + Bootstrap	5
Base de données	PostgreSQL	14+
Gestion de versions	Git / GitHub	Latest
Moteur de templates	Twig (Symfony natif)	Latest

Architecture générale

Backend: Symfony 7 fournit les contrôleurs et services pour la gestion des banques, clients, comptes et transactions.

Frontend: Twig (moteur de templates Symfony natif) avec Bootstrap pour une interface responsive et accessible.

Base de données: PostgreSQL pour la persistance des données avec support des transactions ACID et intégrité référentielle.

Déploiement: Docker Compose pour un environnement reproductible (Symfony + Nginx + PostgreSQL).

8. Modèle de données (4 Tables)

Table: BANQUE

- **id** (INT, PK) - Identifiant unique
- **nom** (VARCHAR, 100) - Nom de la banque
- **email** (VARCHAR, 100, UNIQUE) - Email de contact
- **mot_de_passe** (VARCHAR, 255) - Mot de passe chiffré (Argon2)
- **téléphone** (VARCHAR, 20) - Numéro de téléphone
- **statut** (ENUM: 'actif', 'inactif') - État de la banque
- **date_création** (TIMESTAMP) - Date d'inscription
- **rôle** (VARCHAR) - Toujours 'ROLE_BANQUE'

Table: CLIENT

- **id** (INT, PK) - Identifiant unique
- **nom** (VARCHAR, 100) - Nom du client
- **prénom** (VARCHAR, 100) - Prénom du client
- **email** (VARCHAR, 100, UNIQUE) - Email du client
- **mot_de_passe** (VARCHAR, 255) - Mot de passe chiffré (Argon2)
- **téléphone** (VARCHAR, 20) - Numéro de téléphone
- **rôle** (ENUM: 'ROLE_CLIENT', 'ROLE_ADMIN') - Rôle dans le système
- **statut** (ENUM: 'actif', 'inactif') - État du client
- **date_création** (TIMESTAMP) - Date d'inscription
- **banque_id** (INT, FK → Banque) - Banque associée (NULL si admin)

Table: COMPTE

- **id** (INT, PK) - Identifiant unique
- **numéro_compte** (VARCHAR, 50, UNIQUE) - Numéro IBAN ou compte
- **type** (ENUM: 'courant', 'épargne', 'titre') - Type de compte
- **solde** (DECIMAL(15,2)) - Solde actuel (≥ 0 , NOT NULL)
- **devise** (VARCHAR, 3) - Code devise (EUR, USD, etc.)
- **statut** (ENUM: 'actif', 'inactif') - État du compte
- **date_création** (TIMESTAMP) - Date d'ouverture
- **client_id** (INT, FK → Client, NOT NULL) - Propriétaire du compte

Table: TRANSACTION

- **id** (INT, PK) - Identifiant unique
- **montant** (DECIMAL(15,2)) - Montant de la transaction
- **type** (ENUM: 'débit', 'crédit') - Type d'opération

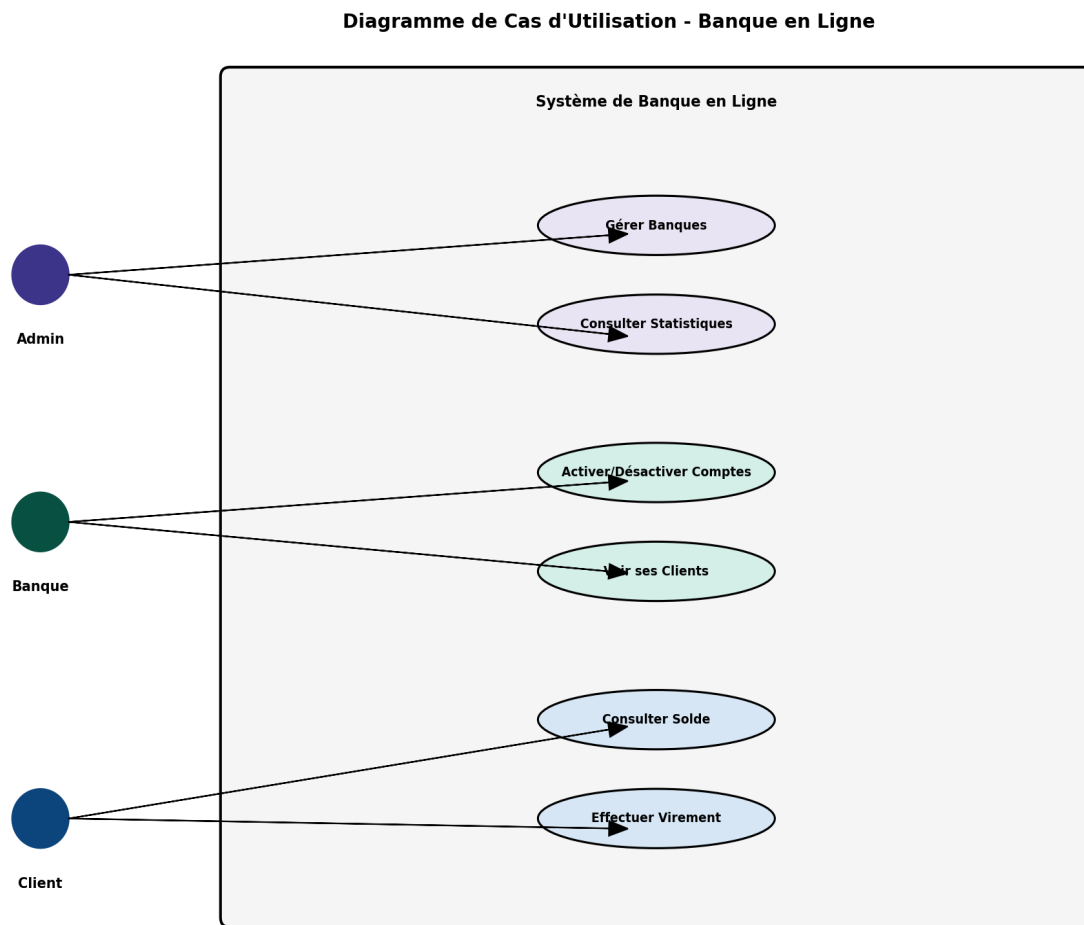
- **libellé** (VARCHAR, 255) - Description de l'opération
- **frais** (DECIMAL(15,2), DEFAULT 0) - Frais appliqués
- **date_transaction** (TIMESTAMP) - Date/heure exacte
- **compte_source_id** (INT, FK → Compte) - Compte d'origine
- **compte_destination_id** (INT, FK → Compte) - Compte destinataire
- **statut** (ENUM: 'succès', 'échoué') - Résultat de la transaction

Relations entre les tables

- Banque (1) ---crée---> (n) Client
- Client (1) ---possède---> (n) Compte
- Compte (1) ---génère---> (n) Transaction
- Transaction relie 2 Comptes: source et destination

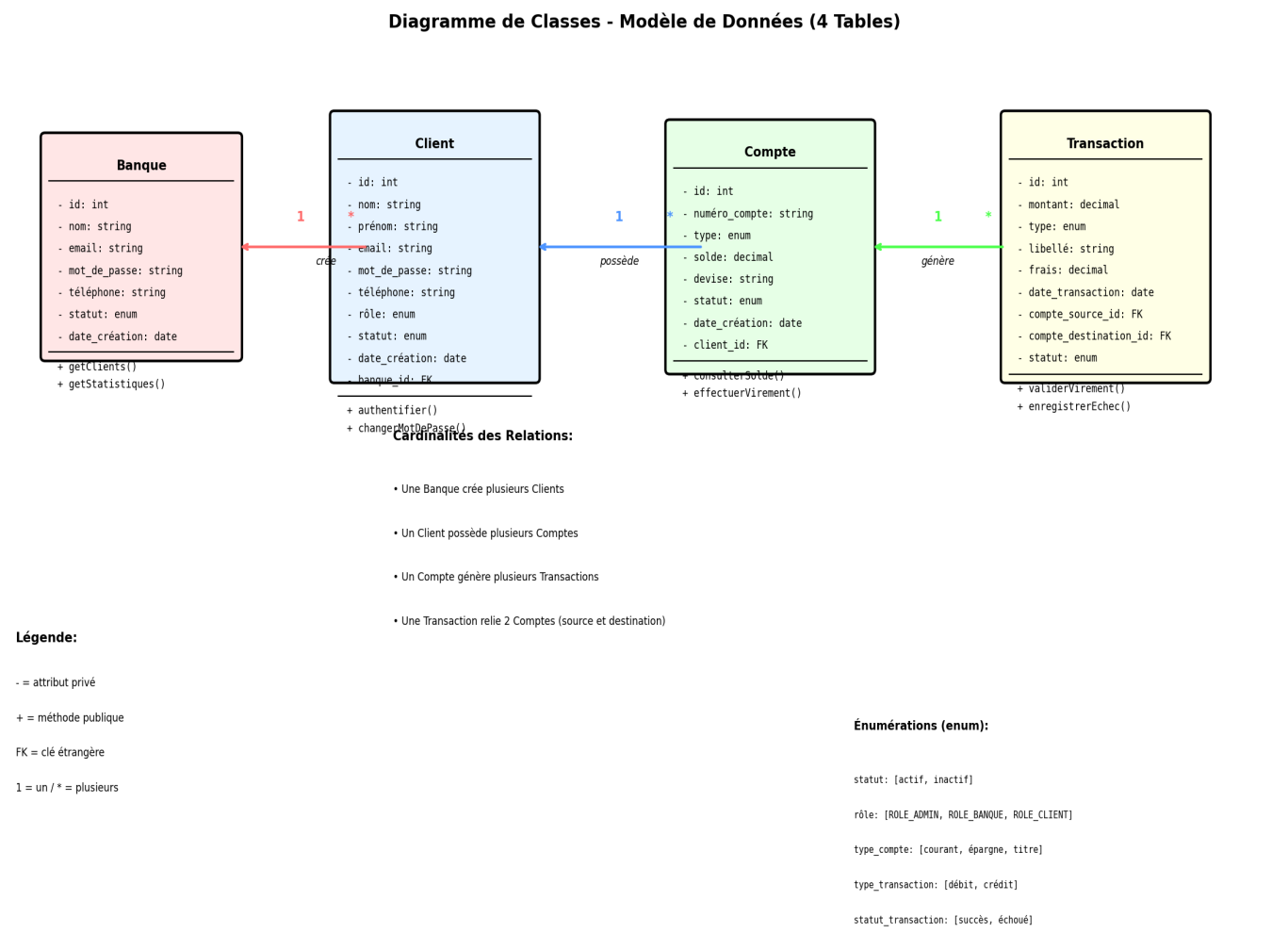
9. Diagrammes UML

9.1 Diagramme de cas d'utilisation



Description: Ce diagramme montre les trois acteurs (Admin, Banque, Client) et leurs interactions avec le système.

9.2 Diagramme de classes UML (Modèle de données)



10. Livrables

- ✓ Cahier des charges (ce document)
- ✓ Diagramme de cas d'utilisation UML
- ✓ Diagramme de classes UML (4 tables)
- ✓ Code source (Symfony 7 + Twig + Bootstrap + PostgreSQL)
- ✓ Script de création de la base de données (migrations Doctrine ORM)
- ✓ Documentation technique (README.md)
- ✓ Docker Compose pour environnement de test
- ✓ Rapport final de réalisation
- ✓ Présentation PowerPoint de soutenance

Critères d'acceptation

- Tous les cas d'utilisation sont fonctionnels
- Les règles métier (R1-R4) sont correctement implémentées
- La sécurité (authentification/autorisation) fonctionne
- Le modèle de données à 4 tables est respecté
- L'application est responsive et compatible mobile
- La base de données est correctement structurée avec intégrité référentielle